Instructions to run:

In the root directory run:
docker compose up --build

Note: A server/.env file is intentionally included in the repository with pre-filled values for convenience. This is a deliberate decision to make running the app as easy as possible — in a production environment, secrets would never be committed to version control.

1. **Can you highlight the parts of the application that are likely to be performance bottlenecks when the user base grows to, say, 10 million users? How would you solve them (you don't need to solve them in code, just outlining and explaining the strategy to solve them is sufficient).**
    - Database
      I'm currently using SQLite, which is a good option for quick MVPs, however in production this would be a problem since SQLite doesn't support concurrent connections. The solution is to migrate to another DBMS such as PostgreSQL, which is well suited for production systems.

    - Database indexes
      In this app we query the transactions table based on different columns such as userId, date, account, and category. Currently these are expensive operations because the database scans the full table on every query. The solution is to create indexes on those columns to speed up these operations.

    - Pagination
      When a user retrieves their transactions, they're getting all records at once. For a user with very few transactions and an app with a small user base (like this example) that is fine, but when the number of users grows or there are users with thousands of transactions, it would be too much data to load into memory on every request. The solution is to implement pagination to avoid hitting memory limits.

2. If a customer wants to send 100k transactions on a daily basis, which changes would we need?
   The current way we handle bulk requests is not efficient. With a large number of transactions to insert it could cause HTTP timeouts, memory issues on the server, and a locked database for a long time (minutes, potentially). The primary solution is to migrate the DBMS as suggested above, and additionally change the bulk import from a synchronous task to an asynchronous one. Instead of the endpoint waiting for all the data to be saved in the database, the endpoint would only create a job in a queue (e.g. Bull), and Bull along with Redis would take care

of finishing the required tasks asynchronously. This approach increases reliability and reduces the latency experienced by the user.